

System Architecture — Pet Profiles

Companion Pet Module (owner-managed pets) — PawGuard Platform

1. Scope

This module manages pets registered directly by their owners (e.g. “my dog Max”), distinct from the shelter's Dog Master Profile module which tracks rescued/admitted dogs. It covers pet records, vet clinic linkage, medical history, QR-based safety tags, and appointment scheduling.

2. Component Overview

Layer	Component	Responsibility
Client	Next.js customer site / mobile apps	Owner-facing pet registration, QR scan landing page
Client	React admin portal	Staff/vet clinic management views
API	FastAPI companion_pet router	Validates requests, enforces RBAC permission codes
Service	companion_pet service layer	Business rules: ownership checks, tag rotation, appointment conflicts
Data	PostgreSQL (companion_pets + related tables)	System of record
Cache/Rate limit	Redis	Rate limiting on public QR scan endpoint
Storage	S3 (pawguard-media bucket)	Medical record file uploads (presigned URLs)
Async	ARQ worker + Redis queue	Appointment reminder delivery, notification emails

3. Request Flow — Register a Pet + Generate Safety Tag

- Owner authenticates → POST /companion-pets (companion_pet:create) → record inserted, owner_id = current user
- Owner requests a QR safety tag → POST /companion-pets/{id}/safety-tag → backend generates random token, stores only its hash, returns raw token once
- Frontend renders the raw token as a QR image pointing to a public scan URL
- Anyone scanning the tag → POST /safety-tag/scan → rate-limited, hash-matched, returns pet's emergency-safe info only (no owner PII)

4. Request Flow — Medical Record Upload

- Vet clinic staff request an upload slot → backend issues a presigned S3 URL
- Client uploads the file directly to S3 (never through the API server)
- Client confirms the upload → backend verifies the object exists → creates the MedicalRecord row linked to pet, clinic, and uploading staff member

5. Security Boundaries

- Every write endpoint requires a specific permission code (companion_pet:create/update/delete, vet_clinic:manage, appointment:manage, safety_tag:manage)
- Ownership is enforced at the service layer: a pet's owner_id must match the requester unless the requester holds a staff/admin permission
- All deletes are soft deletes (deleted_at timestamp) — no record is physically removed
- Public safety-tag scan endpoint never exposes owner identity, address, or phone number